| | |
|---|---|
| **Document Version** | 1.0 |
| **Status** | Draft — Prototype Scope (No AI Agents) |
| **Date** | August 4, 2026 |
| **Standard Reference** | IEEE 830 / ISO/IEC/IEEE 29148 |
| **Classification** | Internal / Graduation Project |

# Table of Contents

# 1. Introduction

## 1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for **SecFlow**, an automated DevSecOps platform that ingests a developer's source-code repository, executes it in an isolated sandbox, runs a suite of automated security scans (SAST, DAST, dependency, and secrets scanning), aggregates and normalizes the findings into a report, and — upon approval — builds and deploys the application to AWS behind a load balancer with live monitoring. This document is intended for the development team, security engineers, cloud engineers, project supervisors, and quality assurance reviewers.

## 1.2 Scope

SecFlow (prototype phase, no autonomous AI agents) covers the full pipeline from user onboarding to live, monitored deployment:

- User registration, authentication, and project/organization management.
- GitHub repository intake, validation, and cloning.
- Isolated, containerized sandbox execution of the target project.
- Automated security scanning: SAST (Semgrep, Bandit), Dependency scanning (Trivy, OWASP Dependency-Check), Secrets scanning (Gitleaks), and DAST for running applications.
- Vulnerability aggregation, normalization, CVE/CWE/CVSS enrichment, and severity classification.
- PDF/HTML report generation and artifact storage.
- Containerized deployment to AWS ECS behind an Application Load Balancer.
- Post-deployment monitoring, logging, alerting, and analytics dashboards.

Out of scope for this phase: autonomous AI-driven auto-remediation agents, policy-as-code enforcement, and multi-cloud support — these are captured as future enhancements in Section 11.

## 1.3 Definitions, Acronyms & Abbreviations

| Term | Definition |
|---|---|
| SAST | Static Application Security Testing — analyzes source code without executing it. |
| DAST | Dynamic Application Security Testing — analyzes a running application for vulnerabilities. |
| CVE / CWE | Common Vulnerabilities and Exposures / Common Weakness Enumeration — public vulnerability and weakness identifiers. |
| CVSS | Common Vulnerability Scoring System — numeric severity score (0–10) for a vulnerability. |
| ECS / ECR / ALB | AWS Elastic Container Service / Elastic Container Registry / Application Load Balancer. |
| RBAC | Role-Based Access Control. |
| SRS | Software Requirements Specification. |
| Sandbox | An isolated, ephemeral Docker container used to safely clone and execute untrusted code. |

## 1.4 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications.
- OWASP Top 10 (2021/2025) and OWASP Dependency-Check documentation.
- Semgrep, Bandit, Trivy, and Gitleaks official documentation.
- AWS ECS, ALB, ECR, RDS, and CloudWatch documentation.
- SecFlow architecture diagrams (Appendix, Figures 1–3).

## 1.5 Document Overview

Section 2 describes the product at a high level. Section 3 presents the system architecture with the supplied diagrams. Sections 4 and 5 detail frontend and backend requirements respectively. Section 6 documents the database design. Sections 7–9 cover interfaces, non-functional requirements, and deployment. Sections 10–12 cover team roles, future roadmap, and appendices.

# 2. Overall Description

## 2.1 Product Perspective

SecFlow is a new, self-contained, cloud-native SaaS platform composed of a Next.js frontend, an ASP.NET Core API gateway, and a set of independent .NET microservices communicating through RabbitMQ and a shared PostgreSQL data store. It is not a modification of an existing system. The platform is designed to scale from a single-developer prototype to a multi-tenant enterprise offering.

## 2.2 Product Functions (Summary)

- Secure user/organization account management with JWT-based authentication.
- GitHub repository connection, cloning, and branch tracking.
- On-demand sandboxed project builds.
- Parallel multi-tool security scanning with unified findings.
- Automated PDF/HTML report generation with CVE/CWE/CVSS enrichment.
- One-click containerized deployment to AWS with health monitoring.
- Real-time notifications, audit logging, and analytics dashboards.

## 2.3 User Classes and Characteristics

| User Class | Description | Technical Level |
|---|---|---|
| Developer | Registers projects, connects repositories, triggers scans, reviews findings. | Medium–High |
| Security Analyst | Reviews findings, triages severity, downloads reports, tracks remediation. | High |
| DevOps / Cloud Engineer | Manages deployments, AWS infrastructure, and monitoring. | High |
| Organization Admin | Manages members, roles, billing, and organization-wide settings. | Medium |
| System Administrator | Manages platform configuration, audit logs, and system health. | High |

## 2.4 Operating Environment

- **Client:** Modern evergreen browsers (Chrome, Edge, Firefox, Safari) on desktop and tablet.
- **Server:** Dockerized microservices orchestrated on AWS ECS (Fargate/EC2 launch type), running Linux containers.
- **Database:** PostgreSQL 15+ (AWS RDS in production, local/Dockerized in development).
- **Messaging:** RabbitMQ for asynchronous background job processing.
- **Cache:** Redis for session cache, rate limiting, and job-status caching.

## 2.5 Design and Implementation Constraints

- Backend services must be implemented in ASP.NET Core (C#), frontend in Next.js/React.
- All untrusted repository code must execute only inside isolated, network-restricted sandbox containers.
- Security scanning tools (Semgrep, Bandit, Trivy, Gitleaks) must be open-source and license-compatible.
- All persisted secrets (API keys, credentials) must be stored via a secrets manager, never in plaintext.
- The prototype phase excludes autonomous AI remediation agents (see Section 11).

## 2.6 Assumptions and Dependencies

- Users have a valid GitHub account and grant repository read access via OAuth or personal access token.
- AWS account and IAM permissions are provisioned prior to deployment configuration.
- Third-party scanning tools remain available and compatible with the versions integrated at build time.

# 3. System Architecture

SecFlow's architecture is organized into three complementary views: the end-to-end prototype workflow, the detailed microservice/system architecture, and the AWS deployment architecture. Each is presented below.

## 3.1 High-Level Prototype Workflow

The diagram below illustrates the seven-stage pipeline a project moves through, from user registration to a live, monitored application, along with the shared infrastructure (RabbitMQ, Redis, PostgreSQL, Object Storage) and supporting reference panels for the microservices layer, technology stack, team roles, and environments.

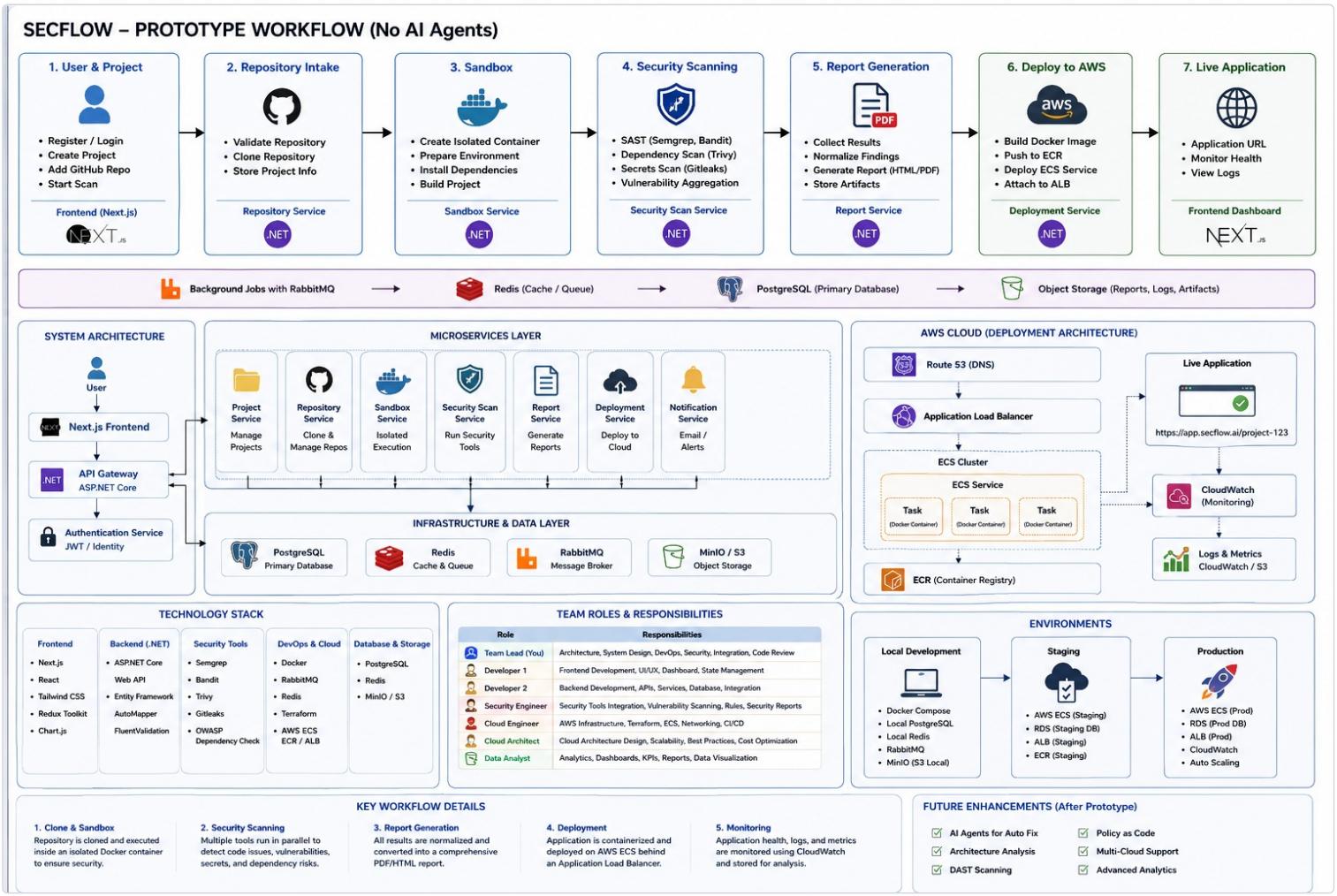


*Figure 1 — SecFlow Prototype Workflow: end-to-end pipeline, microservices layer, infrastructure, technology stack, team roles, and environments.*

### 3.1.1 Pipeline Stages

| # | Stage | Description | Owning Service |
|---|---|---|---|
| 1 | User & Project | Register/login, create project, add GitHub repo, start scan. | Project Service (Next.js frontend) |
| 2 | Repository Intake | Validate repository, clone repository, store project info. | Repository Service (.NET) |
| 3 | Sandbox | Create isolated container, prepare environment, install dependencies, build project. | Sandbox Service (.NET) |
| 4 | Security Scanning | SAST (Semgrep, Bandit), dependency scan (Trivy), secrets scan (Gitleaks), vulnerability aggregation. | Security Scan Service (.NET) |
| 5 | Report Generation | Collect results, normalize findings, generate HTML/PDF report, store artifacts. | Report Service (.NET) |
| 6 | Deploy to AWS | Build Docker image, push to ECR, deploy ECS service, attach to ALB. | Deployment Service (.NET) |
| 7 | Live Application | Application URL, health monitoring, log viewing. | Frontend Dashboard (Next.js) |

## 3.2 Detailed Component Architecture

The following diagram expands the system into its constituent layers: user-facing clients, the API Gateway, the core microservices, pipeline stages, execution environment, storage layer, background services, and the AWS deployment topology.

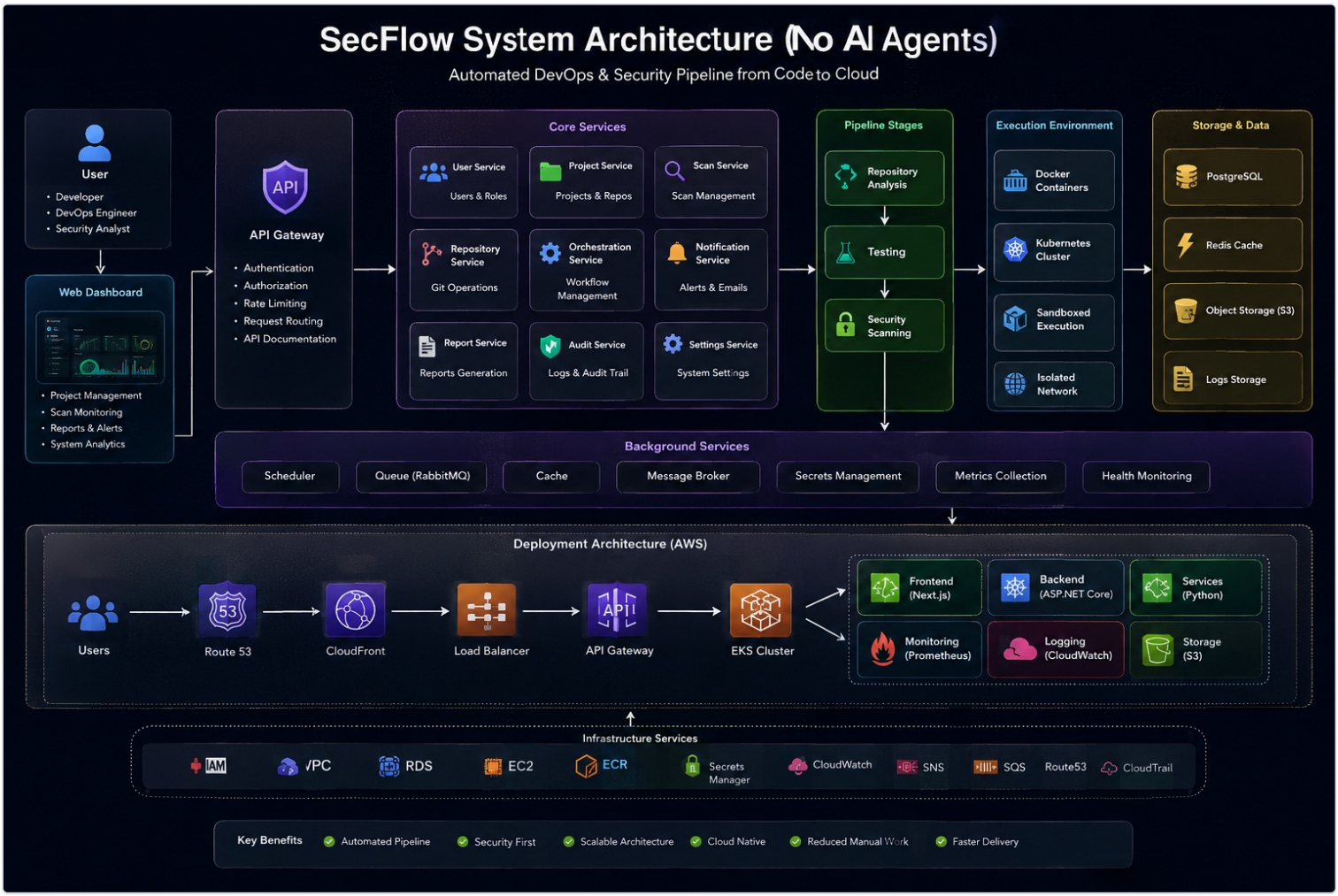


*Figure 2 — SecFlow detailed system architecture: API Gateway, core services, pipeline stages, execution environment, storage, background services, and AWS deployment.*

## 3.3 Architectural Principles

- **Microservices:** each domain capability (users, projects, repositories, scanning, orchestration, notifications, reports, audit, settings) is an independently deployable service behind the API Gateway.
- **API Gateway:** single entry point handling authentication, authorization, rate limiting, request routing, and API documentation (OpenAPI/Swagger).
- **Asynchronous processing:** long-running work (scans, builds, deployments) is queued via RabbitMQ and processed by background workers, with status polled/pushed to the frontend.
- **Isolation by design:** all cloned repository code executes inside sandboxed, network-restricted Docker containers on an isolated network segment.
- **Observability:** Prometheus-style metrics, centralized logging (CloudWatch/S3), and health checks are first-class citizens of every service.

# 4. Frontend Requirements

## 4.1 Technology Stack

| Layer | Technology |
|---|---|
| Framework | Next.js (React) |
| State Management | Redux Toolkit |
| Styling | Tailwind CSS |
| Data Visualization | Chart.js |
| Auth Handling | JWT stored in secure, httpOnly cookies; refresh-token rotation |

## 4.2 Pages & Modules

| Page / Module | Description |
|---|---|
| Authentication | Login, registration, password reset, email verification. |
| Dashboard | Organization/project overview, recent scans, KPI widgets (open findings by severity, deployments this week). |
| Project Management | Create/edit/delete projects, connect GitHub repository, manage branches. |
| Scan Monitoring | Live scan status (queued/running/completed/failed), per-tool progress, cancel/retry. |
| Findings & Reports | Filterable findings table (severity, CWE, CVE, tool), finding detail view, PDF/HTML report download. |
| Deployment Console | Trigger deployment, view ECS task status, application URL, rollback. |
| Monitoring & Logs | Application health, CloudWatch metrics charts, log viewer/search. |
| Alerts & Notifications | In-app notification center, email alert preferences. |
| Organization & Team | Member invitations, role assignment (RBAC), organization settings. |
| Analytics | Trend charts for vulnerabilities over time, scan duration, deployment frequency. |
| Audit Log | Searchable, paginated log of user and system actions. |

## 4.3 Frontend Functional Requirements

| ID | Requirement | Priority |
|---|---|---|
| **FE-01** | The system shall provide a registration and login form with client-side validation and server-side error display. | High |
| **FE-02** | The system shall allow a user to add a GitHub repository via URL or OAuth-based repository picker. | High |
| **FE-03** | The system shall display real-time (poll or WebSocket-based) scan progress with per-stage status indicators. | High |
| **FE-04** | The system shall render findings in a sortable, filterable table with severity color-coding (Critical/High/Medium/Low). | High |
| **FE-05** | The system shall allow users to download generated reports as PDF or view them as HTML in-browser. | Medium |
| **FE-06** | The system shall provide a one-click "Deploy" action with confirmation dialog and live deployment status. | High |
| **FE-07** | The system shall display application health, CPU/memory metrics, and recent logs for deployed services. | Medium |
| **FE-08** | The system shall support role-based UI rendering (Admin, Developer, Security Analyst, Viewer). | Medium |
| **FE-09** | The system shall show toast/in-app notifications for scan completion, deployment status, and alerts. | Low |
| **FE-10** | The system shall be responsive across desktop and tablet viewports (min. width 768px). | Medium |

## 4.4 UI/UX Requirements

- Consistent design system built on Tailwind CSS utility classes with a defined color palette for severity levels.
- Dark-mode support for the monitoring/logs views (matching Figure 3 dashboard style).
- Accessibility: WCAG 2.1 AA color contrast and keyboard navigability for primary workflows.
- Skeleton loaders for all asynchronous data fetches (scans, findings, metrics).

# 5. Backend Requirements

## 5.1 Technology Stack

| Layer | Technology |
|---|---|
| API Framework | ASP.NET Core Web API |
| ORM | Entity Framework Core |
| Object Mapping | AutoMapper |
| Validation | FluentValidation |
| Message Broker | RabbitMQ |
| Cache | Redis |
| Primary Database | PostgreSQL |
| Object Storage | MinIO (local/staging) / AWS S3 (production) |
| Security Scanners | Semgrep, Bandit, Trivy, Gitleaks, OWASP Dependency-Check |
| Containerization | Docker; Terraform for infrastructure-as-code |
| Cloud | AWS ECS, ECR, ALB |

## 5.2 Microservices Overview

| Service | Responsibility |
|---|---|
| User Service | Registration, authentication (JWT/Identity), profile management, RBAC roles. |
| Project Service | CRUD for projects, organization/project linkage. |
| Repository Service | GitHub validation, cloning, branch/commit tracking. |
| Sandbox Service | Provision isolated Docker containers, install dependencies, build project. |
| Security Scan Service | Orchestrates SAST/DAST/dependency/secrets tools; aggregates raw tool output. |
| Orchestration Service | Coordinates multi-step workflows (clone → sandbox → scan → report → deploy) via RabbitMQ. |
| Report Service | Normalizes findings, enriches with CVE/CWE/CVSS, generates PDF/HTML reports. |
| Deployment Service | Builds Docker images, pushes to ECR, deploys to ECS, attaches to ALB. |
| Notification Service | Email and in-app alerts for scan/deployment events. |
| Audit Service | Immutable audit trail of user and system actions. |
| Settings Service | System and organization-level configuration. |
| CI/CD Service | Automates build, test, security self-scan, image publishing, and progressive deployment of SecFlow's own codebase across environments. |

## 5.3 Detailed Service Specifications & Workflows

This section specifies, for every microservice, its purpose, precise responsibilities, step-by-step processing workflow, primary API surface, inputs/outputs, data ownership, dependencies, and failure-handling behavior. Services communicate synchronously (HTTP/REST via the API Gateway) for request/response operations and asynchronously (RabbitMQ events) for long-running, multi-step workflows.

## 5.3.1 User Service

ASP.NET Core · PostgreSQL · Redis

### PURPOSE

Owns identity: registration, authentication, session/token lifecycle, profile data, and role assignment used by every other service for authorization decisions.

### KEY RESPONSIBILITIES

- User registration with email verification and password hashing (bcrypt/Argon2).
- Login and issuance of short-lived JWT access tokens plus long-lived refresh tokens.
- Password reset flow (time-limited, single-use tokens).
- Role and permission management (Owner, Admin, Developer, Security Analyst, Viewer) at the organization and project scope.
- Publishing `UserRegistered`, `UserRoleChanged` events for downstream services (Notification, Audit).

### WORKFLOW — REGISTRATION & LOGIN

1. Client submits registration form (email, password, name) to `POST /api/v1/auth/register`.
2. Service validates input (FluentValidation), checks email uniqueness, hashes password.
3. A verification token is generated and an event is published; Notification Service sends the verification email.
4. User confirms email via `GET /api/v1/auth/verify?token=...`, account status becomes `Active`.
5. On `POST /api/v1/auth/login`, credentials are verified; a JWT access token (15–30 min TTL) and refresh token (7–30 day TTL, stored hashed in Redis) are issued.
6. Subsequent requests present the JWT via `Authorization: Bearer`; the API Gateway validates signature and expiry on every call.
7. On expiry, the client calls `POST /api/v1/auth/refresh`; the refresh token is rotated (old token invalidated, new pair issued).

### KEY ENDPOINTS

**POST** /auth/register | **POST** /auth/login | **POST** /auth/refresh | **POST** /auth/logout | **GET** /users/{id} | **PATCH** /users/{id}/role

### DATA OWNED

`users`, `organization_members`, `roles`, refresh-token blacklist (Redis).

> **Failure handling:** invalid credentials return generic 401 (no user-enumeration); five consecutive failed logins trigger a temporary account lockout with exponential backoff; expired/blacklisted refresh tokens force full re-authentication.

## 5.3.2 Project Service

ASP.NET Core · PostgreSQL

### PURPOSE

Manages the lifecycle of organizations and projects — the top-level containers that group repositories, scans, deployments, and monitoring data.

### KEY RESPONSIBILITIES

- CRUD operations for organizations and projects with ownership/membership enforcement.
- Project status tracking (`Pending`, `Scanning`, `Completed`, `Failed`, `Deploying`, `Running`, `Stopped`).
- Aggregating project-level summary data (latest scan, finding counts, deployment status) for dashboard widgets.

### WORKFLOW — PROJECT CREATION

1. Authenticated user submits `POST /api/v1/projects` with name and organization ID.
2. Service verifies the caller has `Developer` role or above within the organization.
3. A new project record is created with status `Pending`; a project-scoped audit entry is written.
4. Response includes the project ID, which the frontend uses to navigate to the repository-connection step.
5. On any downstream event (`ScanStarted`, `ScanCompleted`, `DeploymentSucceeded`), Project Service updates the cached project status consumed by the dashboard.

### KEY ENDPOINTS

**POST** /projects | **GET** /projects/{id} | **GET** /organizations/{id}/projects | **PATCH** /projects/{id} | **DELETE** /projects/{id}

### DATA OWNED

`organizations`, `projects`.

> **Failure handling:** deletion is soft (sets `deleted_at`) to preserve historical scan/deployment records; concurrent status updates use optimistic concurrency (row version) to avoid lost updates.

## 5.3.3 Repository Service

ASP.NET Core · GitHub API · Git CLI

### PURPOSE

Connects SecFlow to source-control providers, validates repository access, and performs the initial clone that seeds a sandbox execution.

### KEY RESPONSIBILITIES

- GitHub OAuth handshake and personal-access-token validation.
- Repository existence, visibility, and branch validation before accepting a scan request.
- Cloning the target branch/commit into a working volume mounted by the Sandbox Service.
- Tracking commit hash, branch, and last-synced timestamp for re-scan triggers.

### WORKFLOW — REPOSITORY INTAKE

1. User submits a GitHub URL (or selects a repo via the OAuth picker) on `POST /api/v1/repositories`.
2. Service calls the GitHub API to confirm the repository exists and the token has read access.
3. If validation fails (private/inaccessible/not found), a 422 error with a specific reason is returned to the frontend.
4. On success, a `repositories` record is created (status `Validated`) and a `RepositoryReady` event is published to RabbitMQ.
5. The Orchestration Service consumes the event and requests sandbox provisioning from the Sandbox Service.
6. The actual `git clone --depth 1 --branch <branch>` is executed inside the sandbox container (not on the host), writing into an isolated volume.
7. Clone success/failure and elapsed time are recorded; failures publish a `RepositoryCloneFailed` event, halting the pipeline and notifying the user.

### KEY ENDPOINTS

**POST** /repositories | **GET** /repositories/{id} | **POST** /repositories/{id}/resync | **POST** /webhooks/github

### DATA OWNED

`repositories`.

> **Failure handling:** transient GitHub API errors are retried with exponential backoff (max 3 attempts); webhook signature is verified (HMAC-SHA256) to prevent spoofed push events from triggering unauthorized scans.

## 5.3.4 Sandbox Service

ASP.NET Core · Docker Engine API

### PURPOSE

Provides ephemeral, isolated, resource-limited execution environments in which untrusted repository code is cloned, its dependencies installed, and the project built — with no access to production systems or the internet beyond approved package registries.

### KEY RESPONSIBILITIES

- Provisioning a new Docker container per scan job from a hardened base image matching the project's detected language/runtime.
- Enforcing CPU, memory, disk, process-count, and execution-time limits (cgroup constraints).
- Restricting outbound network access to an allow-list (package registries only) via network policy.
- Installing dependencies (`npm install`, `pip install`, `dotnet restore`, etc.) and building the project.
- Tearing down and destroying the container and its volume once scanning is complete, regardless of success/failure.

### WORKFLOW — SANDBOX PROVISIONING & BUILD

1. Consumes `RepositoryReady` event; selects a base image based on detected project type (language manifest inspection).
2. Creates an isolated container with a dedicated network namespace, no host mounts, and resource quotas defined per plan tier.
3. Clones the repository into the container's working directory (see 5.3.3, step 6).
4. Installs dependencies using the appropriate package manager, capturing full stdout/stderr logs.
5. Builds/compiles the project where applicable (e.g. `npm run build`, `dotnet build`).
6. Publishes `SandboxReady` on success or `SandboxBuildFailed` (with captured logs) on failure.
7. On pipeline completion (or timeout), the container and its ephemeral volume are force-destroyed; no state persists between scans.

### KEY ENDPOINTS

**POST** /sandboxes | **GET** /sandboxes/{id}/status | **GET** /sandboxes/{id}/logs | **DELETE** /sandboxes/{id}

### DATA OWNED

`sandboxes` (container_id, status, resource usage snapshot).

> **Failure handling:** a hard execution-time ceiling (default 15 minutes) force-kills runaway builds; any container that exceeds resource quotas is OOM-killed by the Docker runtime and reported as `SandboxBuildFailed` rather than left running.

## 5.3.5 Security Scan Service

ASP.NET Core · Semgrep · Bandit · Trivy · Gitleaks

### PURPOSE

Orchestrates the execution of all security-scanning tools against the built sandbox, collects their raw output, and hands normalized results to the Report Service.

### KEY RESPONSIBILITIES

- Selecting applicable scanners based on detected language(s)/frameworks (e.g. Bandit for Python, Semgrep multi-language rulesets).
- Launching SAST, dependency, and secrets scans **in parallel** inside the sandbox to minimize total pipeline time.
- Optionally running DAST against a temporarily deployed instance of the application for dynamic analysis.
- Parsing each tool's native output format (SARIF/JSON) into raw result rows per scan job.
- Tracking per-tool status, start/end time, and exit code independently, so one tool's failure doesn't block the others.

### WORKFLOW — PARALLEL MULTI-TOOL SCANNING

1. Consumes `SandboxReady` event and creates a parent `scan_job` record.
2. Dispatches four parallel worker tasks via RabbitMQ: SAST (Semgrep/Bandit), Dependency (Trivy/OWASP Dependency-Check), Secrets (Gitleaks), and — if the project is web-runnable — DAST against a short-lived preview instance.
3. Each worker runs its tool inside the sandbox container, streaming logs and writing raw SARIF/JSON output to a scan-scoped storage path.
4. As each tool completes, a `ToolScanCompleted` event (with tool name and result path) is published; the parent job tracks completion count.
5. Once all dispatched tools report completion (or timeout), a `ScanJobCompleted` event is published for the Report Service to consume.
6. If a tool crashes, its status is marked `Failed` with the captured error, but the overall job proceeds with the remaining tool results (partial-success model).

### KEY ENDPOINTS

| | | | | |
|---|---|---|---|---|
| **POST** /scans | **GET** /scans/{id} | **GET** /scans/{id}/tools | **POST** /scans/{id}/cancel | **POST** /scans/{id}/retry |

### DATA OWNED

`scan_jobs`, `sast_results`, `dast_results`, `dependency_results` (raw, pre-normalization).

> **Failure handling:** per-tool timeout (default 10 minutes) marks that tool `Failed` without aborting siblings; if all tools fail, the parent job is marked `Failed` and the pipeline halts before report generation.

## 5.3.6 Orchestration Service

ASP.NET Core · RabbitMQ · Redis (state)

### PURPOSE

Acts as the workflow coordinator (saga orchestrator) that drives a project through the full pipeline — clone → sandbox → scan → report → (optional) deploy — ensuring each stage begins only after its predecessor succeeds.

### KEY RESPONSIBILITIES

- Maintaining pipeline state per project run (current stage, timestamps, terminal status) in Redis for fast reads.
- Subscribing to lifecycle events from every stage service and issuing the next command message.
- Implementing compensating actions on failure (e.g. destroy sandbox, mark project `Failed`, notify user) so no stage is left dangling.
- Exposing a single "pipeline status" read model the frontend polls/subscribes to, instead of querying each service individually.

### WORKFLOW — SAGA COORDINATION

1. Receives `ScanRequested` command (from Project Service, triggered by "Start Scan") and initializes pipeline state.
2. Issues `ValidateRepository` command to Repository Service; awaits `RepositoryReady` or `RepositoryCloneFailed`.
3. On success, issues `ProvisionSandbox` to Sandbox Service; awaits `SandboxReady` or `SandboxBuildFailed`.
4. On success, issues `RunScan` to Security Scan Service; awaits `ScanJobCompleted`.
5. Issues `GenerateReport` to Report Service; awaits `ReportGenerated`.
6. Marks the pipeline `Completed` and notifies the user; if the user requests deployment, issues `StartDeployment` to the Deployment Service as a separate, user-gated stage.
7. On any stage failure, executes compensation (tear down sandbox, mark project `Failed`), publishes `PipelineFailed`, and stops — no subsequent stage is invoked.

### KEY ENDPOINTS

| | | |
|---|---|---|
| **GET** /pipelines/{projectId}/status | **POST** /pipelines/{projectId}/start | **POST** /pipelines/{projectId}/cancel |

### DATA OWNED

Pipeline state cache (Redis); no primary relational data — it is a coordinator over other services' data.

> **Failure handling:** every command is idempotent and carries a correlation ID, so a redelivered RabbitMQ message cannot double-trigger a stage; a stalled stage (no event within its SLA) is timed out and treated as a failure with compensation.

## 5.3.7 Report Service

ASP.NET Core · CVE/CWE feeds · PDF renderer

### PURPOSE

Transforms raw, tool-specific scan output into a single normalized set of findings enriched with vulnerability intelligence, then renders a human-readable report.

### KEY RESPONSIBILITIES

- Parsing SARIF/JSON output from each tool into a unified `finding` schema (title, severity, file, line, rule ID).
- Enriching dependency findings with CVE identifiers, mapping to CWE weakness categories, and attaching CVSS scores.
- De-duplicating overlapping findings reported by multiple tools for the same underlying issue.
- Computing severity roll-ups (counts by Critical/High/Medium/Low) for dashboard KPIs.
- Rendering the final report as both HTML (in-browser view) and PDF (downloadable artifact) and storing both in object storage.

### WORKFLOW — NORMALIZATION & REPORT GENERATION

1. Consumes `ScanJobCompleted`; loads all raw per-tool results for the scan job.
2. Maps each raw result into the unified `findings` table row, tagging its source tool.
3. For dependency findings, cross-references known CVE databases; resolves associated CWE and CVSS score.
4. Runs de-duplication (same file + line range + rule family) to avoid inflating counts.
5. Aggregates severity counts and generates the HTML report from a template, then renders it to PDF.
6. Uploads both artifacts to object storage (MinIO/S3) and records their paths in the `reports` table.
7. Publishes `ReportGenerated`, which the Orchestration Service and Notification Service both consume.

### KEY ENDPOINTS

**GET** /reports/{scanId} | **GET** /reports/{scanId}/pdf | **GET** /findings?scanId= | **GET** /findings/{id}

### DATA OWNED

`findings`, `reports`, `cve`, `cwe`, `cvss` reference tables.

> **Failure handling:** if CVE/CWE enrichment lookups are temporarily unavailable, the finding is still stored with tool-native severity and flagged for later re-enrichment rather than blocking report generation.

## 5.3.8 Deployment Service

ASP.NET Core · Docker · AWS SDK (ECR/ECS/ALB)

### PURPOSE

Packages the scanned project as a container image and deploys it to AWS, exposing it behind a load balancer with health checks.

### KEY RESPONSIBILITIES

- Building a production Docker image from the project source.
- Pushing the image to Amazon ECR with an immutable, commit-hash-based tag.
- Creating/updating the ECS task definition and service to run the new image.
- Registering the service with the target ALB target group and configuring health-check paths.
- Providing rollback to the previous known-good task definition on failed health checks.

### WORKFLOW — BUILD, PUSH & DEPLOY

1. User triggers deployment from the dashboard (only enabled once the pipeline is `Completed` and, per policy, no unresolved Critical findings block it).
2. Service builds a Docker image tagged with the repository commit hash.
3. Image is pushed to the project's ECR repository.
4. A new ECS task definition revision is registered referencing the new image.
5. The ECS service is updated to the new task definition; ECS performs a rolling deployment, starting new tasks before draining old ones.
6. The ALB target group health check polls the new tasks; once healthy, traffic shifts fully to the new revision.
7. On health-check failure within the deployment window, ECS/Deployment Service automatically rolls back to the previous task definition and publishes `DeploymentFailed`.
8. On success, `DeploymentSucceeded` is published with the live application URL, consumed by Notification Service and the frontend.

### KEY ENDPOINTS

**POST** /deployments | **GET** /deployments/{id} | **POST** /deployments/{id}/rollback | **GET** /deployments/{id}/logs

### DATA OWNED

`deployments`, `aws_resources`, `load_balancers`, `application_urls`, `deployment_logs`, `environments`.

> **Failure handling:** deployments are atomic from the user's perspective — a failed rollout never leaves the ALB pointing at an unhealthy task set; all rollbacks are automatic and logged.

## 5.3.9 Notification Service

ASP.NET Core · SMTP · RabbitMQ

### PURPOSE

Delivers timely, relevant alerts to users through email and in-app channels based on pipeline and monitoring events.

### KEY RESPONSIBILITIES

- Subscribing to domain events ( `ScanJobCompleted` , `ReportGenerated` , `DeploymentSucceeded/Failed` , `AlertRaised` ).
- Rendering event-specific email templates and dispatching via the configured SMTP/email provider.
- Writing in-app notification records surfaced in the dashboard's notification center.
- Respecting per-user notification preferences (channel and event-type opt-in/out).

### WORKFLOW — EVENT-DRIVEN ALERTING

1. Consumes a domain event from RabbitMQ (e.g. `DeploymentFailed` ).
2. Looks up the affected project's members and each member's notification preferences.
3. For each eligible member, creates an in-app notification row and, if email is enabled, queues a templated email.
4. Email delivery is attempted with retry-on-transient-failure; permanent failures are logged, not retried indefinitely.

### KEY ENDPOINTS

**GET** /notifications | **PATCH** /notifications/{id}/read | **GET/PATCH** /notification-preferences

### DATA OWNED

`notifications` , `notification_preferences` , `alerts` .

> **Failure handling:** a down email provider does not block the pipeline — in-app notifications are always written first and independently of email delivery success.

## 5.3.10 Audit Service

ASP.NET Core · Append-only store

### PURPOSE

Maintains a tamper-evident, immutable record of every state-changing action across the platform for compliance, forensics, and troubleshooting.

### KEY RESPONSIBILITIES

- Subscribing to all domain events published by other services and persisting a normalized audit record.
- Recording actor (user/service), action, target entity, timestamp, and before/after payload where applicable.
- Providing searchable, paginated audit-log access to Admins.

### WORKFLOW — AUDIT CAPTURE

1. Every service publishes a lightweight audit event alongside its domain event when it performs a state change.
2. Audit Service consumes the event and appends an immutable row (no update/delete API is exposed).
3. Admins query `GET /audit-logs` with filters (actor, action type, date range, project).

### KEY ENDPOINTS

**GET** /audit-logs | **GET** /audit-logs/{id}

### DATA OWNED

`audit_logs` (append-only).

> **Failure handling:** audit writes are treated as at-least-once with idempotency keys, so a redelivered event produces one record, not duplicates.

## 5.3.11 Settings Service

ASP.NET Core · PostgreSQL

### PURPOSE

Centralizes system- and organization-level configuration that other services read at runtime.

### KEY RESPONSIBILITIES

- Managing organization-level policy toggles (e.g. "block deploy on Critical findings").
- Managing scan tool enablement per project (which scanners run by default).
- Exposing a cached configuration read API consumed by the Orchestration and Deployment Services.

### WORKFLOW — CONFIGURATION READ/WRITE

1. Admin updates a setting via `PATCH /settings/{orgId}` .
2. Service validates the change against the schema for that setting key and persists it.
3. A `SettingsChanged` event invalidates the Redis-cached copy consumed by dependent services.

### KEY ENDPOINTS

**GET** /settings/{orgId} | **PATCH** /settings/{orgId}

### DATA OWNED

`organization_settings` , `system_settings` .

> **Failure handling:** dependent services fall back to safe defaults if the settings cache is unavailable (fail-closed for security-relevant policies, e.g. deploy-blocking).

### 5.3.12 CI/CD Service

GitHub Actions · Docker · Terraform · AWS ECS/ECR

**PURPOSE**

Automates the build, test, self-scan, and progressive deployment of SecFlow's **own** microservices and frontend across environments. This is distinct from the Deployment Service in Section 5.3.8, which deploys *end users' scanned projects*; the CI/CD Service governs how the SecFlow platform itself ships.

**KEY RESPONSIBILITIES**

- Triggering automated pipelines on every push, pull request, and tagged release across all SecFlow repositories (frontend, API Gateway, and each microservice).
- Running linting, unit tests, and integration tests for each affected service.
- Dogfooding SecFlow's own Security Scan Service (SAST/dependency/secrets) against the platform's codebase before any merge is allowed.
- Building and tagging Docker images, publishing them to Amazon ECR with immutable, commit-hash-based tags.
- Applying Terraform plans for infrastructure changes with mandatory human approval before apply.
- Orchestrating progressive promotion of new builds: Local → Staging (automatic) → Production (manual gate).
- Executing post-deploy smoke tests and automatically rolling back a release that fails them.

**WORKFLOW — CONTINUOUS INTEGRATION (PER COMMIT/PR)**

1. Developer opens a pull request; GitHub Actions triggers the CI workflow for every service touched by the diff.
2. Static analysis and linting run (ESLint/Prettier for Next.js, StyleCop/Roslyn analyzers for .NET).
3. Unit and integration tests execute against ephemeral test containers (PostgreSQL, Redis, RabbitMQ spun up via Docker Compose in the CI runner).
4. SecFlow's own Security Scan Service is invoked against the changed code (SAST + dependency + secrets) as a required check — this is the platform scanning itself.
5. Code coverage is computed and compared against the minimum threshold (fails the build if below target).
6. On success, all checks are marked green and the PR becomes eligible for merge; a failing check blocks merge via branch protection rules.

**WORKFLOW — CONTINUOUS DELIVERY (ON MERGE TO MAIN / TAG)**

1. Merge to `main` triggers the CD workflow for each changed service.
2. A Docker image is built and tagged with the Git commit SHA, then pushed to the service's ECR repository.
3. Terraform plan is generated for any infrastructure diffs and posted to the PR/pipeline log for review.
4. The new image is automatically deployed to the **Staging** ECS cluster; ECS performs a rolling update behind the Staging ALB.
5. Automated smoke tests (health endpoint checks, critical-path API calls) run against Staging; failures halt promotion and alert the on-call engineer.
6. On Staging success, a deployment gate requires manual approval (Team Lead / Cloud Engineer) before promoting to **Production**.
7. Upon approval, the identical image (never rebuilt) is deployed to the Production ECS cluster using the same rolling-update strategy described in Section 5.3.8.
8. CloudWatch alarms monitor the new revision for a defined bake time (e.g. 10 minutes); if error rates or latency exceed thresholds, the pipeline automatically triggers rollback to the previous task definition.

**PIPELINE STAGES SUMMARY**

**1** Lint & Static Analysis · **2** Unit / Integration Tests · **3** Security Self-Scan · **4** Build & Push Image · **5** Terraform Plan · **6** Deploy → Staging · **7** Smoke Tests · **8** Manual Approval Gate · **9** Deploy → Production · **10** Post-Deploy Monitoring / Auto-Rollback

**DATA OWNED**

Pipeline run history and deployment metadata ( `cicd_runs` , `cicd_stage_results` ); links to the same `deployments` and `deployment_logs` tables used by the Deployment Service for environment-promotion records.

> **Failure handling:** any failed stage halts the pipeline for that service only (independent per-service pipelines); Production deploys are never triggered automatically — the manual approval gate is a hard requirement; failed smoke tests or breached CloudWatch alarms during the bake window trigger automatic rollback with no human action required.

## 5.4 End-to-End Pipeline Sequence

The table below summarizes the full event sequence across services for a single **runtime "scan and deploy" run** (a user's project moving through SecFlow), showing how the asynchronous, event-driven handoffs described above chain together. For how SecFlow's *own* codebase is built, tested, and shipped, see Section 10, CI/CD Pipeline & Workflow.

| Step | Trigger / Event | Acting Service | Result / Next Event |
|---|---|---|---|
| 1 | User clicks "Start Scan" | Project Service → Orchestration Service | Pipeline initialized; `ScanRequested` |
| 2 | `ScanRequested` | Repository Service | Validate + clone; `RepositoryReady` |
| 3 | `RepositoryReady` | Sandbox Service | Provision container, install deps, build; `SandboxReady` |
| 4 | `SandboxReady` | Security Scan Service | Parallel SAST/DAST/Dependency/Secrets scans; `ScanJobCompleted` |
| 5 | `ScanJobCompleted` | Report Service | Normalize, enrich, render PDF/HTML; `ReportGenerated` |
| 6 | `ReportGenerated` | Notification Service | Email + in-app alert sent to project members |
| 7 | User clicks "Deploy" | Deployment Service | Build image, push to ECR, deploy to ECS/ALB; `DeploymentSucceeded` |
| 8 | Continuous | Monitoring (CloudWatch) → Notification Service | Health metrics tracked; alerts raised on threshold breach |
| — | Every step above | Audit Service | Immutable audit record appended in parallel |

## 5.5 Backend Functional Requirements

| ID | Requirement | Priority |
|---|---|---|
| **BE-01** | The system shall authenticate users via JWT bearer tokens with configurable expiry and refresh-token rotation. | High |
| **BE-02** | The system shall validate repository URLs and reject inaccessible or malformed repositories before cloning. | High |
| **BE-03** | The system shall clone repositories exclusively inside network-isolated, resource-limited sandbox containers. | High |

| | | |
|---|---|---|
| **BE-04** | The system shall run SAST, dependency, and secrets scans in parallel and record per-tool status and duration. | **High** |
| **BE-05** | The system shall normalize all tool-specific findings into a unified schema (severity, CWE, CVE, CVSS, file/line). | **High** |
| **BE-06** | The system shall generate a PDF and HTML report per scan job and persist it to object storage. | **Medium** |
| **BE-07** | The system shall build a Docker image from the project, push it to ECR, and deploy it as an ECS service. | **High** |
| **BE-08** | The system shall attach deployed services to an Application Load Balancer with health checks. | **Medium** |
| **BE-09** | The system shall publish scan and deployment lifecycle events to RabbitMQ for asynchronous processing. | **Medium** |
| **BE-10** | The system shall record every state-changing action in an immutable audit log with actor, timestamp, and payload. | **Medium** |
| **BE-11** | The system shall enforce role-based authorization on every API endpoint via the API Gateway. | **High** |
| **BE-12** | The system shall rate-limit API requests per user/organization to prevent abuse. | **Low** |

## 5.6 API Design Principles

- RESTful resource-oriented endpoints (e.g. `/api/v1/projects/{id}/scans`) documented via OpenAPI/Swagger.
- Consistent envelope for success/error responses, including correlation IDs for tracing across services.
- Idempotent write operations for scan and deployment triggers to safely handle retries.
- Webhook support for CI/CD-triggered scans (e.g. on push/PR events from GitHub).

## 5.7 AWS Deployment Architecture

The diagram below shows the target deployment topology: user traffic enters via Route 53, CloudFront, and an Application Load Balancer, is routed through the API Gateway to an EKS/ECS cluster hosting the frontend, backend, and Python-based scanning services, with monitoring (Prometheus), logging (CloudWatch), and storage (S3) as supporting infrastructure.


*Figure 2 (reference) — Deployment Architecture (AWS) section: Route 53 → CloudFront → Load Balancer → API Gateway → EKS Cluster, with Frontend, Backend, Services, Monitoring, Logging, and Storage.*

# 6. Database Design (Entity-Relationship Diagram)

SecFlow's relational schema is normalized to 3NF and organized around six domains: identity & organizations, projects & repositories, sandbox execution, security scanning & findings, reporting, and deployment & monitoring. Every core entity uses a UUID primary key and includes standard audit columns ( `created_at` , `updated_at` , `deleted_at` for soft deletes) where applicable.

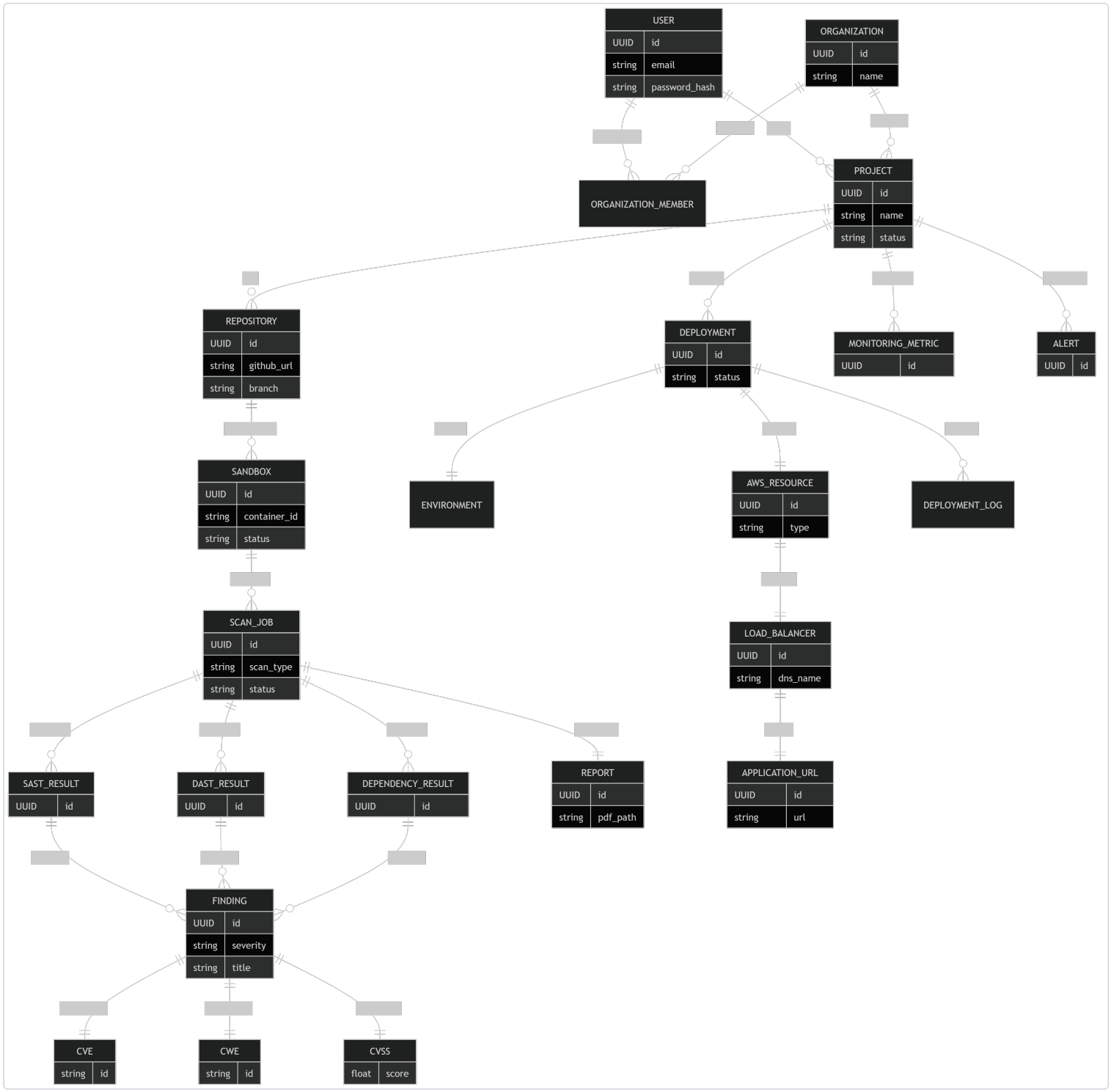


*Figure 3 — SecFlow Entity-Relationship Diagram: Users, Organizations, Projects, Repositories, Sandboxes, Scan Jobs, Findings (with CVE/CWE/CVSS), Reports, Deployments, and AWS/Monitoring entities.*

## 6.1 Core Entities

| Entity | Key Attributes | Relationships |
|---|---|---|
| User | id, email, password_hash | Many-to-many with Organization via Organization_Member |
| Organization | id, name | Has many Projects; has many Members |
| Project | id, name, status | Belongs to Organization; has many Repositories, Deployments, Monitoring Metrics, Alerts |
| Repository | id, github_url, branch | Belongs to Project; has many Sandboxes |
| Sandbox | id, container_id, status | Belongs to Repository; has many Scan Jobs |
| Scan Job | id, scan_type, status | Produces SAST/DAST/Dependency Results and a Report |
| Finding | id, severity, title | Linked to CVE, CWE, CVSS; sourced from a scan result |
| Report | id, pdf_path | Belongs to Scan Job |

| | | |
|---|---|---|
| Deployment | id, status | Belongs to Project; has an Environment, AWS Resource, Deployment Log |
| AWS Resource | id, type | Belongs to Deployment; has a Load Balancer and Application URL |

# 7. External Interface Requirements

## 7.1 User Interfaces

Web-based responsive dashboard (Next.js) as described in Section 4, accessed via HTTPS on desktop and tablet browsers.

## 7.2 Hardware Interfaces

None directly; the system runs entirely on virtualized/cloud compute (AWS ECS) and requires no dedicated hardware.

## 7.3 Software Interfaces

| Interface | Purpose |
|---|---|
| GitHub API / OAuth | Repository access, cloning, webhook events. |
| Semgrep / Bandit | Static application security testing. |
| Trivy / OWASP Dependency-Check | Dependency and container image vulnerability scanning. |
| Gitleaks | Secrets detection in source code and commit history. |
| AWS ECS / ECR / ALB / RDS / CloudWatch | Container orchestration, image registry, load balancing, database, monitoring. |
| SMTP / Email Provider | Transactional emails and alert notifications. |

## 7.4 Communication Interfaces

- HTTPS/TLS 1.2+ for all client-server and service-to-service external communication.
- AMQP for internal asynchronous messaging via RabbitMQ.
- WebSocket or Server-Sent Events (optional) for real-time scan/deployment status updates.

# 8. Non-Functional Requirements

| Category | Requirement |
|---|---|
| Performance | The API Gateway shall respond to standard CRUD requests within 300ms (p95) under nominal load. Scan jobs execute asynchronously and shall not block API responsiveness. |
| Scalability | Each microservice shall scale horizontally and independently via ECS service auto-scaling based on CPU/queue depth. |
| Security | All secrets stored in a secrets manager; sandbox containers run with no outbound network access except to approved package registries; RBAC enforced on all endpoints; audit logging of all sensitive actions. |
| Reliability / Availability | Target 99.5% uptime for the API Gateway and dashboard during the prototype phase; automatic ECS task restarts on health-check failure. |
| Maintainability | Services shall follow clean architecture with clear separation of controllers, services, and repositories; consistent logging and error-handling middleware. |
| Portability | All services are containerized (Docker) and configuration-driven, enabling deployment to any container orchestrator. |
| Usability | New users shall be able to onboard, connect a repository, and trigger a first scan within 5 minutes without external documentation. |
| Data Retention | Scan artifacts and reports retained for a minimum of 90 days; soft-deleted records retained per compliance policy. |

# 9. Deployment & Infrastructure (AWS)

## 9.1 Environments

| Environment | Composition |
|---|---|
| Local Development | Docker Compose, local PostgreSQL, local Redis, local RabbitMQ, MinIO (local S3). |
| Staging | AWS ECS (Staging), RDS (Staging DB), ALB (Staging), ECR (Staging). |
| Production | AWS ECS (Production), RDS (Production DB), ALB (Production), CloudWatch, Auto Scaling. |

## 9.2 Deployment Flow

1. Deployment Service builds a Docker image from the validated, scanned project.
2. Image is pushed to Amazon ECR (Elastic Container Registry).
3. An ECS service is created/updated to run the image as one or more tasks within an ECS Cluster.
4. The ECS service is registered with an Application Load Balancer target group, with health checks configured.
5. Route 53 DNS resolves the application's public hostname to the ALB.
6. CloudWatch collects logs and metrics; alerts are raised via the Notification Service on threshold breaches.

## 9.3 Infrastructure Services

Supporting AWS infrastructure includes IAM (access control), VPC (network isolation), RDS (managed PostgreSQL), EC2 (compute for self-managed components), ECR (image registry), Secrets Manager (credential storage), CloudWatch (monitoring/logging), SNS/SQS (notifications/queuing), Route 53 (DNS), and CloudTrail (API audit logging).

# 10. CI/CD Pipeline & Workflow

This section documents how the **SecFlow platform itself** is built, tested, security-checked, and shipped — distinct from the runtime scan-and-deploy pipeline SecFlow runs on behalf of its users (Sections 3.1 and 5.4). The CI/CD pipeline is owned by the CI/CD Service described in Section 5.3.12 and is implemented with GitHub Actions, Docker, Terraform, and AWS ECS/ECR, following a trunk-based development model with mandatory automated checks and a manual production gate.

## 10.1 CI/CD Philosophy

- **Trunk-based development:** short-lived feature branches merge into `main` via pull request; no long-running branches.
- **Shift-left security:** SecFlow scans its own code with its own Security Scan Service on every pull request — the platform dogfoods itself before any code reaches Staging.
- **Independent per-service pipelines:** each microservice (User, Project, Repository, Sandbox, Security Scan, Orchestration, Report, Deployment, Notification, Audit, Settings) and the Next.js frontend has its own pipeline, triggered only when its own code changes (monorepo path-filtering) or its shared libraries change.
- **Immutable artifacts:** the exact image built and scanned in CI is the same image promoted through Staging and Production — nothing is rebuilt between environments.
- **Progressive delivery:** every change flows Local → Staging (automatic) → Production (manual approval), never directly to Production.
- **Fail fast, roll back automatically:** failed tests, failed security gates, failed smoke tests, or breached post-deploy health alarms all halt or reverse a release without waiting on a human.

## 10.2 CI/CD Workflow Diagram

The end-to-end flow from a developer's commit to a live production release is illustrated below.

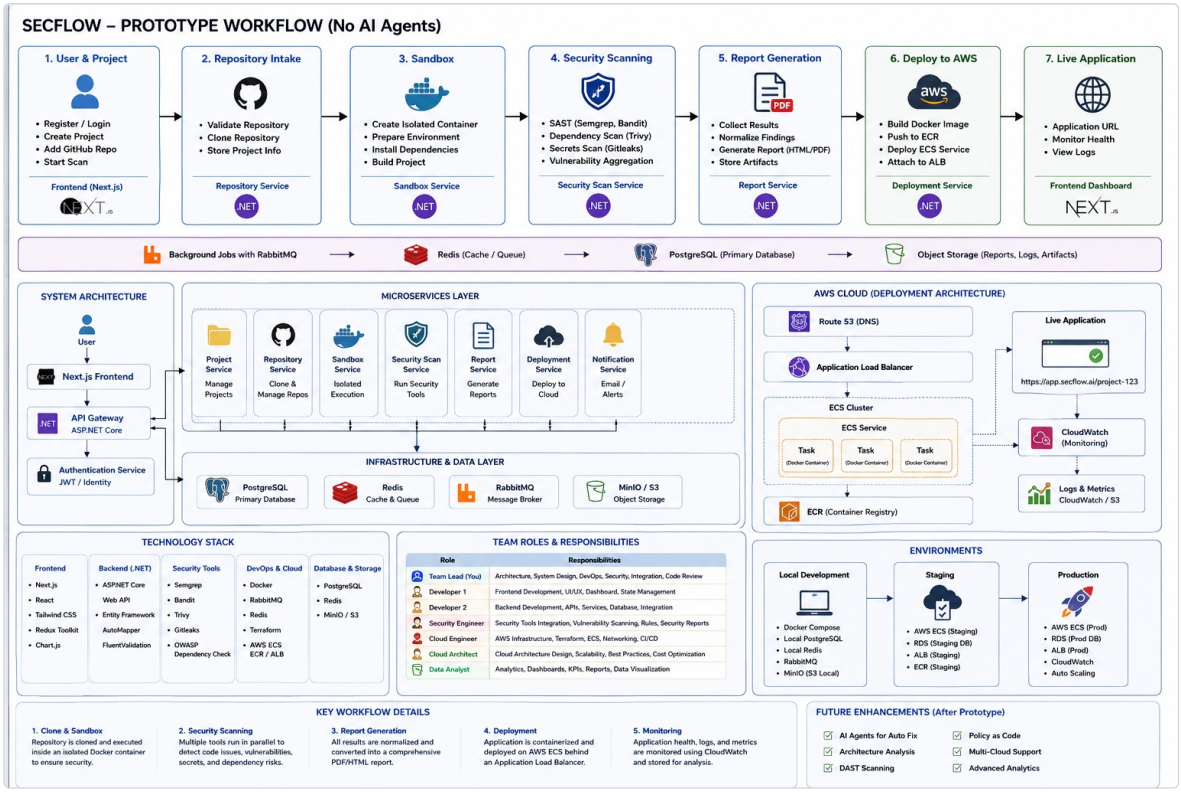


*Figure 4 — Reference technology stack and background-services infrastructure (RabbitMQ, Redis, PostgreSQL, Object Storage) that underpin every CI/CD pipeline run.*

| Stage | Name | Trigger | Actions |
|---|---|---|---|
| 1 | Source | Push / Pull Request | Developer pushes to a feature branch; opens a PR against `main`. |
| 2 | Lint & Static Analysis | PR opened/updated | ESLint/Prettier (Next.js); StyleCop/Roslyn analyzers (.NET services). |
| 3 | Build | After lint passes | `dotnet build` per service; `next build` for frontend. |
| 4 | Unit & Integration Tests | After build | xUnit/NUnit tests; ephemeral PostgreSQL/Redis/RabbitMQ via Docker Compose in the runner; Jest/React Testing Library for frontend. |
| 5 | Security Self-Scan | After tests pass | SecFlow's own Security Scan Service (Semgrep, Bandit, Trivy, Gitleaks) runs against the diff as a required PR check. |
| 6 | Code Review & Merge Gate | All checks green | Required peer review + branch protection; merge to `main` blocked until lint, tests, and security scan all pass. |
| 7 | Build & Publish Image | Merge to `main` | Docker image built, tagged with Git commit SHA, pushed to the service's ECR repository. |
| 8 | Infrastructure Plan | After image publish | Terraform plan generated for any infra changes; diff posted to the pipeline log for review. |
| 9 | Deploy to Staging | Automatic | ECS rolling update on the Staging cluster behind the Staging ALB. |
| 10 | Smoke Tests | Post-Staging-deploy | Automated health-endpoint and critical-path API checks; failure halts promotion and alerts on-call. |
| 11 | Manual Approval Gate | Staging verified | Team Lead / Cloud Engineer approves promotion to Production. |
| 12 | Deploy to Production | On approval | Same immutable image deployed via ECS rolling update behind the Production ALB. |
| 13 | Post-Deploy Bake & Monitor | Continuous, ~10 min window | CloudWatch alarms watch error rate/latency; breach triggers automatic rollback to the prior task definition. |

## 10.3 Environment Promotion Model

| Environment | Promotion Trigger | Approval Required | Purpose |
|---|---|---|---|
| Local Development | Developer-initiated (Docker Compose) | None | Rapid iteration and debugging on a developer's machine. |
| CI (ephemeral) | Every PR / push | None (automated gates) | Lint, test, and security-scan the change in isolation. |
| Staging | Merge to `main` | None (fully automatic) | Integration verification against a production-like AWS environment. |
| Production | Manual promotion after Staging smoke tests pass | Yes — Team Lead / Cloud Engineer | Live, customer-facing environment; protected by an approval gate and automatic rollback. |

## 10.4 Illustrative Pipeline Definition

The following is a representative, simplified GitHub Actions workflow outline for a single backend microservice, showing the required checks (lint, test, security self-scan) that gate merges, and the separate deployment workflow that promotes through Staging and Production.

```
name: ci.yml — triggered on: [pull_request, push to main]
jobs:
  lint:        run dotnet format --verify-no-changes
  build:       run dotnet build --configuration Release
  test:        run dotnet test --collect:"XPlat Code Coverage"
  security-scan:
    needs: [build]
    run: call SecFlow Security Scan Service API against the changed service
  publish-image:
    needs: [lint, test, security-scan]
    if: github.ref == 'refs/heads/main'
    run: docker build, tag with ${{ github.sha }}, push to ECR

name: cd.yml — triggered on: [workflow_run: ci.yml completed on main]
jobs:
  deploy-staging:
    run: aws ecs update-service --cluster staging --force-new-deployment
  smoke-test-staging:
    needs: [deploy-staging]
    run: health-check + critical-path API tests against Staging URL
  await-approval:
    needs: [smoke-test-staging]
    environment: production   # GitHub "environment" protection rule requires manual reviewer
  deploy-production:
    needs: [await-approval]
    run: aws ecs update-service --cluster production --force-new-deployment
  post-deploy-monitor:
    needs: [deploy-production]
    run: watch CloudWatch alarms for 10 min; rollback task definition on breach
```

## 10.5 Rollback Strategy

- Every ECS deployment is a rolling update that retains the previous task definition revision until the new one is confirmed healthy.
- Automatic rollback triggers: failed ALB health checks during rollout, failed post-deploy smoke tests, or CloudWatch alarm breaches (error rate, latency, CPU/memory) within the bake window.
- Manual rollback is available via `POST /deployments/{id}/rollback` (Deployment Service, Section 5.3.8) for issues detected after the automated bake window closes.
- Database migrations are applied as backward-compatible, additive changes so a service rollback never requires a corresponding database rollback.

# 11. Team Roles & Responsibilities

| Role | Responsibilities |
|---|---|
| Team Lead | Architecture, system design, DevOps, security, integration, code review. |
| Frontend Developer | Frontend development, UI/UX, dashboard, state management. |
| Backend Developer | Backend development, APIs, services, database, integration. |
| Security Engineer | Security tools integration, vulnerability scanning, rules, security reports. |
| Cloud Engineer | AWS infrastructure, Terraform, ECS, networking, CI/CD. |
| Cloud Architect | Cloud architecture design, scalability, best practices, cost optimization. |
| Data Analyst | Analytics, dashboards, KPIs, reports, data visualization. |

## 12. Future Enhancements (Post-Prototype)

- AI agents for automated vulnerability remediation ("Auto-Fix").
- Policy-as-code enforcement for compliance gating (e.g. block deploy on Critical findings).
- Architecture analysis and automated design-review suggestions.
- Multi-cloud deployment support (Azure, GCP) in addition to AWS.
- Expanded DAST scanning coverage for authenticated application flows.
- Advanced analytics: predictive risk scoring and remediation-time forecasting.

# 13. Appendix

## 13.1 Traceability Note

Requirement identifiers (FE-xx for frontend, BE-xx for backend) in this document map directly to backlog items and test cases to be tracked in the project's issue tracker during implementation.

## 13.2 List of Figures



> This SRS reflects the prototype scope explicitly excluding autonomous AI agents, per the source architecture diagrams. Sections 3–10 should be reviewed and versioned alongside future architecture revisions.